

Game_Database.py

```
import mysql.connector as mysql
import pickle

#DatabaseUser,DatabasePassword= "root","root"          # <- Change and Unhash this for
AUTO_SIGN_IN to Database
DatabaseUser= input("Enter Name of User: ")
DatabasePassword= input("Enter the Password: ")
print()

try:
    sql= mysql.connect(host="localhost",user=DatabaseUser,password=DatabasePassword)
    if sql.is_connected():
        print("Connected to MySQL Successfully...")

except mysql.errors.ProgrammingError as e:
    if e.errno == 1045:
        print("Incorrect Username or Password")
        print("Exiting...")
    else:
        print("Unknown Error Occoured")
        print("Exiting...")
    exit()

cur= sql.cursor()

# __Functions__

def Recreate_Database():
    global DatabasePassword
    pasw= input("Enter Password: ")
    if pasw==DatabasePassword:
        cur.execute("DROP DATABASE IF EXISTS GAME_DATA")
        cur.execute("CREATE DATABASE GAME_DATA;")
        sql.commit()
        cur.execute("USE GAME_DATA;")

        cur.execute("CREATE TABLE PLAYER_DATA(" \
        "UID INT UNSIGNED PRIMARY KEY," \
        "USERNAME VARCHAR(20) NOT NULL UNIQUE," \
        "PASSWORD VARCHAR(50) NOT NULL," \
        "ACCOUNT VARCHAR(76) NOT NULL UNIQUE," \
        "JOIN_DATE DATE NOT NULL);")

        cur.execute("CREATE TABLE CURRENCY_MANAGE(" \
        "UID INT UNSIGNED PRIMARY KEY," \
        "ACCOUNT VARCHAR(76) NOT NULL UNIQUE," \
        "COINS INT DEFAULT 0," \
        "GEMS INT DEFAULT 0," \
        "MONEY_USED INT DEFAULT 0," \
        "FOREIGN KEY (UID) REFERENCES PLAYER_DATA(UID) ON DELETE CASCADE," \
        "FOREIGN KEY (ACCOUNT) REFERENCES PLÅYER_DATA(ACCOUNT) ON UPDATE CASCADE ON DELETE
CASCADE);")

        cur.execute("CREATE TABLE ITEM_SHOP(" \
        "PACK_ID INT PRIMARY KEY," \
        "ITEM_NAME VARCHAR(20)," \
        "ITEM_AMOUNT INT," \
        "REAL_MONEY INT); ")

        cur.execute("INSERT INTO ITEM_SHOP VALUES" \
        "(1, 'COINS',2000, 180)," \
        "(2, 'COINS',5000, 400)," \
        "(3, 'GEMS',100, 50)," \
        "(4, 'GEMS',250, 115);")

        sql.commit()
```

```

        print("Database Recreated!")

    else:
        print("Incorrect Password")
while True:
    try:
        cur.execute("USE GAME_DATA;")
        print("Connected to Database Successfully...")
        break
    except Exception:
        print("GAME_DATA Database Does Not Exist.")
        print("Recreating Database...")
        Recreate_Database()

def Inplen_Validate(comment,minl=0,maxl=20,dtype="str",brkr=False):
    while True:
        valinp= input(comment)
        if valinp==' ' and brkr:
            return ' '
        if dtype=="int":
            try:
                val= int(valinp)
                if val>=minl and val<=maxl:
                    return val
            except:
                else:
                    print("Integer Input is between",minl,"and",maxl)
        except ValueError:
            print("It is an Integer Input")
            pass
    else:
        if len(valinp)>=minl and len(valinp)<=maxl:
            return valinp
        else:
            print("Input Length is between",minl,"Characters and",maxl,"Characters")

def acc_inp(n=" Account",j=False):
    al["@gmail.com","@hotmail.com","@outlook.com","@icloud.com","@mail.com"]
    while True:
        acc= Inplen_Validate("Enter the{0}: ".format(n),16,76,"str",brkr=j)
        if acc==' ':
            return ' '
        valid= False
        for com in al:
            if com in acc:
                valid= True
                break
        if not valid:
            print("That is not an Account(Format xyz@domain.com)")
            print("Available domain: gmail, hotmail, outlook, icloud, mail")
        if valid:
            return acc

def date_valid():
    retval= "False"
    while retval=="False":
        cur.execute("DROP TABLE IF EXISTS Dtab1;")
        cur.execute("CREATE TABLE Dtab1(SL INT PRIMARY KEY, D DATE);")
        sql.commit()
        try:
            dateinp= Inplen_Validate("Enter Date in Format yyyy-mm-dd: ",10,10)
            cur.execute("INSERT INTO Dtab1 VALUES(0,'{0}')".format(dateinp))
            sql.commit()
            cur.execute("SELECT CASE " \
                "WHEN D > '2010-12-31' AND D < CURDATE() THEN 'T' " \
                "ELSE 'False' " \
                "END AS is_valid " \
                "FROM Dtab1 WHERE SL=0;")
            retval= cur.fetchone()[0]
            cur.execute("DROP TABLE Dtab1;")
            sql.commit()
            retdate= dateinp

```

```

        except Exception:
            retval= "False"
            sql.commit()
    return retdate

def update_object(x,y,z):
    cur.execute("UPDATE PLAYER_DATA SET {0}= '{1}' WHERE UID= {2}".format(x,y,z))
    sql.commit()
    print(x.title(),"Successfully Updated")

def Update_Transaction(packid,uid):
    cur.execute("SELECT ITEM_NAME,ITEM_AMOUNT,REAL_MONEY FROM ITEM_SHOP WHERE
PACK_ID={0}".format(packid))
    tup= cur.fetchone()
    a,b,c= tup[0],tup[1],tup[2]
    cur.execute("UPDATE CURRENCY_MANAGE " \
        "SET MONEY_USED= MONEY_USED+{0}, {1}= {2}+{3} WHERE UID= {4};".format(c,a,a,b,uid))
    sql.commit()
    print("Transaction Succesfull")

def uid_check(ret=1,stu=True,com="Enter the UID(Press Enter to Exit): "):
    while True:
        uid= Inplen_Validate(com,100000,999999,"int",stu)
        cur.execute("SELECT * FROM PLAYER_DATA WHERE UID='{0}'".format(uid))
        data= cur.fetchone()
        if uid=="":
            return uid
        elif data==None:
            print("UID",uid,"Not Found")
        else:
            if ret==1:
                return uid
            else:
                return data

# __Main_Menu__

run= "y"
while run=="y":
    print()
    print("+-----+")
    print("|   Main Menu   |")
    print("+-----+")
    print("<1> Manage Database Operations")
    print("<2> Manage Player Transactions")
    print("<3> Database Backup and Restore")
    print("<4> Recreate GAME_DATA Database")
    print("<5> Restart Program")
    print("<6> Exit Program")
    ch= Inplen_Validate("Enter your Choice: ",1,6,"int")

    while ch==1:
        print()
        print("+-----+")
        print("| Database Operations Menu |")
        print("+-----+")
        print("<1> Enter Data into PLAYER_DATA")
        print("<2> Show Entire PLAYER_DATA TABLE")
        print("<3> Select Specific Data from PLAYER_DATA")
        print("<4> Update PLAYER_DATA Record")
        print("<5> Delete Record from PLAYER_DATA")
        print("<6> Back to Main Menu")

        ch2= Inplen_Validate("Enter your Choice: ",1,6,"int")

        if ch2 == 6:
            break

        if ch2==1:
            uid= Inplen_Validate("Enter the UserID(Press Enter to Exit):
",100000,999999,"int",True)

```

```

        if uid=="":
            print("Data Input Cancelled")
            break
        uname= Inplen_Validate("Enter the Username: ",6,20)
        pasw= Inplen_Validate("Enter the Password: ",4,20)
        acc= acc_inp()
        jdate= date_valid()
        try:
            cur.execute("INSERT INTO PLAYER_DATA
VALUES({0},' {1}', '{2}', '{3}', '{4}').format(uid,uname,pasw,acc,jdate))
            sql.commit()
            print("Record Saved Successfully")
            cur.execute("INSERT INTO CURRENCY_MANAGE
VALUES({0},' {1}', 0,0,0).format(uid,acc))
            sql.commit()
            print("Currency Account Created Automatically!")
        except mysql.errors.IntegrityError as e:
            if e.errno==1062:
                msg= e.msg.lower()
                if "uid" in msg:
                    print("Err: UID already exists.")
                elif "username" in msg:
                    print("Err: Username already exists.")
                elif "account" in msg:
                    print("Err: Account already exists.")
                else:
                    print("Err: Duplicate entry detected.")
            else:
                print("Database Integrity Error")
        except Exception as e:
            print("An Unexpected Error has Occurred")

elif ch2==2:
    cur.execute("SELECT * FROM PLAYER_DATA;")
    table1= cur.fetchall()
    print("  UID      Username      Password      Account      Join_date")
    for i in table1:
        for j in i:
            print(j,end=" ")
        print()

elif ch2==3:
    data= uid_check(2,True,"Enter UID to View(Press Enter to Exit): ")
    if data=="":
        print("Record Viewing Cancelled")
    print("  UID      Username      Password      Account      Join_date")
    for i in data:
        print(i,end=" ")

elif ch2==4:
    upuid= uid_check(1,True,"Enter the UID to Update(Press Enter to Exit): ")
    if upuid=="":
        print("Record Update Cancelled")
        break
    print("What would you like to Update?")
    print("<1> Username")
    print("<2> Password")
    print("<3> Account")
    ch1= Inplen_Validate("Enter your Choice: ",1,3,"int")
    cur.execute("SELECT * FROM PLAYER_DATA")
    tab1= cur.fetchall()
    cur.execute("SELECT * FROM PLAYER_DATA WHERE UID= {0}".format(upuid))
    uid1= cur.fetchone()

    if ch1==1:
        c= 1
        brk= False
        while c>0 and not brk:
            c=0
            upusr= Inplen_Validate("Enter the New Username(Press Enter to Exit):
",4,20,"str",brkr=True)

```

```

        if upusr=='':
            brk= True
            print("Update Cancelled")
            break
        else:
            for i in tabl:
                if i[1]==upusr:
                    print("Username already Exists")
                    c+= 1
    if not brk:
        update_object("USERNAME", upusr, upuid)
        print("Username Updated Successfully")

elif ch1==2:
    curpass= "a"
    uppass= "b"
    brk= False
    while curpass!= uidl[2] and not brk:
        curpass= Inplen_Validate("Enter the Current Password(Press Enter to
Exit): ",4,20,"str",brkr=True)
        if curpass=='':
            print("Update Cancelled")
            break
        elif curpass!= uidl[2]:
            print("Incorrect Password")
        else:
            uppass= Inplen_Validate("Enter the New Password: ",4,20)
            update_object("PASSWORD",uppass,upuid)
            print("Password Updated Successfully")

else:
    upacc= "a"
    c= 1
    brk= False
    while c>0 and not brk:
        c=0
        upacc= acc_inp(" New Account(Press Enter to Exit)",True)
        if upacc=='':
            brk= True
            print("Update Cancelled")
            break
        else:
            for i in tabl:
                if i[3]==upacc:
                    print("Account already Exists")
                    c+= 1
    if not brk:
        update_object("ACCOUNT",upacc,upuid)
        print("Account Updated Successfully")

elif ch2==5:
    uid= uid_check(1,True,"Enter the UserID to Delete(Press Enter to Cancel): ")
    if uid=="":
        print("Deletion Cancelled")
        break
    cur.execute("DELETE FROM PLAYER_DATA WHERE UID= {0}".format(uid))
    sql.commit()
    print("Record with UserID=",uid,"Deleted")

while ch==2:
    print()
    print("+-----+")
    print("| Player Transaction Menu |")
    print("+-----+")
    print("<1> Buy Coins")
    print("<2> Buy Gems")
    print("<3> Convert Gems to Coins")
    print("<4> Coin and Gem Earning through Gameplay")
    print("<5> Show Player Transaction Record and Total Earning")
    print("<6> Back to Main Menu")

```

```

ch2= Inplen_Validate("Enter Choice: ",1,6,"int")

if ch2== 6:
    break

if ch2==1:
    uid= uid_check()
    if uid=="":
        print("Transaction Cancelled")
        break
    print("Select Pack to Buy: ")
    print(" <1> 2000 Coins -> Rs 180.00")
    print(" <2> 5000 Coins -> Rs 400.00")
    pack_ch= Inplen_Validate("Enter Choice: ",1,2,"int")
    if pack_ch==1:
        Update_Transaction(1,uid)
    else:
        Update_Transaction(2,uid)

elif ch2==2:
    uid= uid_check()
    if uid=="":
        print("Transaction Cancelled")
        break
    print("Select Pack to Buy: ")
    print(" <1> 100 Gems -> Rs 50.00")
    print(" <2> 250 Gems -> Rs 115.00")
    pack_ch= Inplen_Validate("Enter Choice: ",1,2,"int")
    if pack_ch==1:
        Update_Transaction(3,uid)
    else:
        Update_Transaction(4,uid)

elif ch2==3:
    uid= uid_check()
    if uid=="":
        print("Gem Conversion Cancelled")
        break
    cur.execute("SELECT GEMS FROM CURRENCY_MANAGE WHERE UID={0}".format(uid))
    avlgem= cur.fetchone()[0]
    gemn= Inplen_Validate("Enter the Amount of Gem to Convert into Coins:
",0,avlgem,"int")
    coinn= 10*gemn
    cur.execute("UPDATE CURRENCY_MANAGE " \
"SET GEMS= GEMS-{0}, COINS= COINS+{1} WHERE UID= {2}".format(gemn,coinn,uid))
    sql.commit()
    print(gemn,"Gems Converted to",coinn,"Coins. Enjoy :)")

elif ch2==4:
    uid= uid_check()
    if uid=="":
        print("User Input Cancelled")
        break
    gplcoin= Inplen_Validate("Enter Coins Earned Through Gameplay: ",0,100000,"int")
    gplgem= Inplen_Validate("Enter Gems Earned Through Gameplay: ",0,30,"int")
    cur.execute("UPDATE CURRENCY_MANAGE " \
"SET GEMS= GEMS+{0}, COINS= COINS+{1} WHERE UID=
{2}".format(gplgem,gplcoin,uid))
    sql.commit()

elif ch2==5:
    cur.execute("SELECT p.UID, p.USERNAME, p.ACCOUNT, c.COINS, c.GEMS, c.MONEY_USED
" \
"FROM PLAYER_DATA p, CURRENCY_MANAGE c " \
"WHERE p.UID = c.UID")
    lst= cur.fetchall()
    print("Full Transaction Record")
    print("UID      Username      Account      Coins      Gems      Spent")
    for i in lst:
        for j in i:
            print(j,end=" ")

```

```

        print()
        cur.execute("SELECT SUM(MONEY_USED) FROM CURRENCY_MANAGE")
        revenue= cur.fetchone()[0]
        print()
        print("Total Money Earned: Rs {}".format(revenue))

while ch==3:
    print()
    print("+-----+")
    print("| Database Backup and Restore Menu |")
    print("+-----+")
    print("<1> Backup Database into File")
    print("<2> Use Backup File to Restore Database")
    print("<3> Back to Main Menu")
    ch2= Inplen_Validate("Enter Choice: ",1,3,"int")

    if ch2==3:
        break

    if ch2==1:
        cur.execute("SELECT * FROM PLAYER_DATA;")
        table1= cur.fetchall()
        cur.execute("SELECT * FROM CURRENCY_MANAGE;")
        table2= cur.fetchall()
        tabl1st= [table1,table2]
        with open(r"Backup.dat", "wb") as file:
            pickle.dump(tabl1st,file)
        print("Database Backed Up!")

    elif ch2==2:
        lst= []
        try:
            floc= input("Enter File Location: ")
            with open(floc,"rb") as file:
                lst= pickle.load(file)
        except EOFError:
            print("File Empty or Corrupted")
            break
        except FileNotFoundError:
            print("File Not Found")
            break
        except PermissionError:
            print("Code Doesnt have Permission to that area")
            print("Run code in Administrator Mode for accessing")
            break

        c1=0
        for i in lst[0]:
            try:
                cur.execute("INSERT INTO PLAYER_DATA
VALUES({0},' {1}',' {2}',' {3}',' {4}').format(i[0],i[1],i[2],i[3],i[4],))
                sql.commit()
            except Exception:
                c1+=1
                print("Common Record Found =",c1)
                print(c1,"Record/s Skipped")

        c2=0
        for i in lst[1]:
            try:
                cur.execute("INSERT INTO CURRENCY_MANAGE
VALUES({0},' {1}',' {2}',' {3}',' {4}').format(i[0],i[1],i[2],i[3],i[4],))
                sql.commit()
            except Exception:
                c2+=1
                print("Common Record Found =",c2)
                print(c2,"Record/s Skipped")

        print("Database Restored through Backup")

    if ch==4:
        print("Are you Sure you want to recreate the Database?")

```

```
print("*you WILL LOOSE your current data")
ch2= Inplen_Validate("Enter y/n: ",0,1)
if ch2=="y":
    Recreate_Database()
    print()
else:
    print("Database Recreation Cancelled")
    print()

elif ch==5:
    run= "y"
    continue
elif ch==6:
    sql.close()
    run= "n"
    print("Thank you for using!")
    continue
else:
    run= "y"
    continue
```